TRELLIS / TECHNICAL DESIGN

System Architecture and Design Baseline

A proposed implementation architecture for the AI-assisted Instagram growth, analysis and content-planning application.

Version	Date	Status
0.1 - proposed	23 October 2026	For technical and Product Owner review

Source requirements: Trellis Business Requirements Document v1.0 and Software Requirements Specification v1.0, both dated 22 October 2026 and marked as consolidated working baselines.
Design intent: Provide a reviewable high-level architecture with clear component responsibilities, data flows, security boundaries, operational behavior, traceability and decisions awaiting feasibility validation.

PROPOSED ARCHITECTURE
Angular / TypeScript frontend · modular Go backend · PostgreSQL system of record · provider-neutral AI and data adapters · controlled background jobs

1. Executive architecture summary

Trellis is an **iPad-first responsive browser application** serving an initial skincare/K-beauty creator while retaining distinct user ownership for future commercialization. The proposal uses a **modular monolith**: one deployable Go application containing explicit feature modules and boundary interfaces, paired with Angular and PostgreSQL.

1.1 Architecture principles

• **Evidence before inference:** measurements and news retain identifiable sources, timestamps and limitations (AI-03/04, DATA-02, DISC-03, NEWS-01).

• **Creator remains in control:** editable drafts, active prohibited terms and an explicit action for every publication (CONT-03/06/07, PLAN-03/04).

• **Responsive without false status:** saved information renders while external work proceeds; save and publication outcomes are honestly represented (PERF-01/03, REL-01/02).

• **Provider independence:** Instagram, topic/news and free-tier AI integrations sit behind replaceable adapters (MAINT-01, COST-01).

• **Security by boundary:** tenant isolation, server-side credentials and explicit Instagram identity confirmation (AUTH-01/08).

• **No premature infrastructure:** do not require microservices, Kubernetes, Redis or paid providers for the initial scope (BRD §3, COST-01).

1.2 Fixed inputs versus proposals

Classification	Architecture input / decision
Requirements baseline	Angular/TypeScript, Go, PostgreSQL; browser app; one linked Instagram account per user; free-tier AI/providers; no unattended publishing.
Proposed implementation	Modular Go monolith with explicit module interfaces, a PostgreSQL-backed job processor and private media storage abstraction.
Unselected / to validate	Auth implementation/vendor, compatible official Meta permissions, competitor/topic coverage, AI providers, media host, quantitative service thresholds, deployment.

IMPORTANT SCOPE CONTROL
Publishing only occurs after review and a user press of Publish for that specific approved version. A planned date or reminder never grants autonomous publishing authority. The Product Owner must acknowledge this change from the earlier feature summary.

2. System context and container design

The browser interacts with the Go API over authenticated HTTPS. The Go application owns business policy, data access and authorization. Remote platforms and AI providers are treated as untrusted external dependencies.

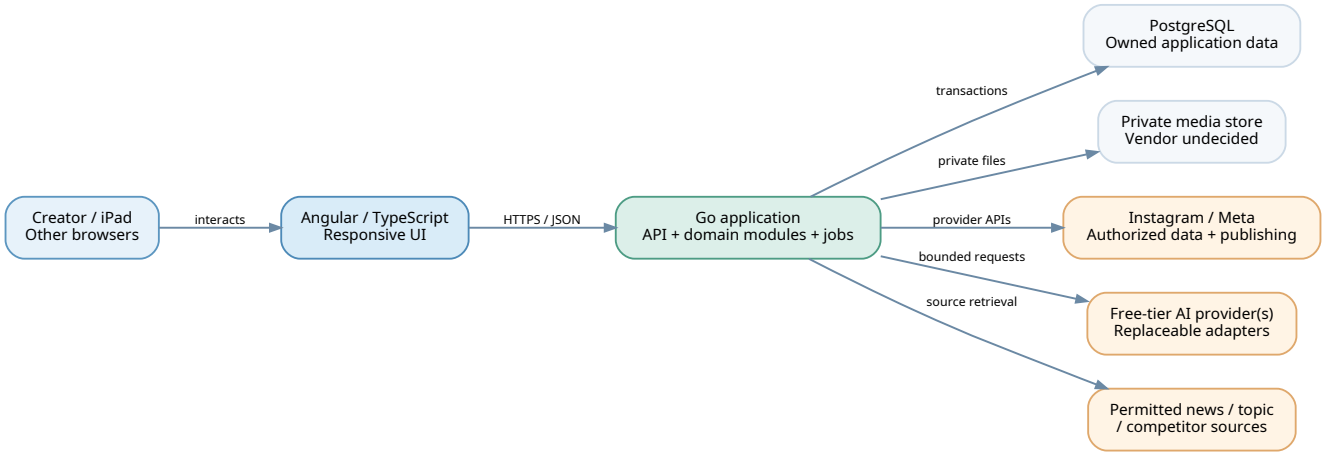


Figure 1. Proposed system context. External-provider feasibility and permission coverage are not yet verified.

3. Go backend component responsibilities

The following components are internal logical modules, not separate independently deployed services. They should communicate through explicit interfaces or application services, with a shared authorization and transaction policy.

Module	Responsibility / required boundaries
Identity & Access	Trellis sign-in, secure sessions, per-user authorization, account ownership and deletion coordination.
Instagram Connection	Official account authorization; confirmation of Instagram identity; token lifecycle, disconnect and collection halt.
Analytics & History	Profile/post metrics, historical observations, comparable calculations, freshness status, session-open analysis and manual refresh.
AI Orchestration	Interpret natural language; select authorized tools; construct evidence context; invoke compatible free providers; validate and label outputs.
Content & Creator Voice	Private media associations, generation inputs, caption style retrieval, editable post packages, prohibited terms, draft versions.
Planner & Publishing	Daily content plans, reminders, approved draft versions, user-triggered publish intents, attempt records and uncertain-outcome resolution.
Competitors & Discovery	Five-latest-post skincare eligibility test, comparable competitor evidence, combined Hot Topics / Niche Ideas / Swiper workflows.
Official Updates	Attributed Instagram algorithm / Meta Edits announcements, publication dates and separation of reported fact from inference.
Jobs & Integration Adapters	Controlled refresh tasks, bounded retries for safe work, APIs for Instagram, AI and compliant external sources.

3.1 Rules for module boundaries

• Only the Identity & Access layer establishes the authenticated Trellis user; each domain operation then enforces resource ownership.

• AI tools may call approved read/query operations but never bypass authorization, directly mutate restrictions or perform publishing.

• Publishing routes through its own module and requires a current explicit approval tied to an immutable draft version.

• Integrations normalize provider-specific response shapes into internal evidence records, preserving provenance and access limitations.

• Background jobs invoke the same domain policies as synchronous API requests; they do not create extra permissions.

4. Conceptual data architecture

PostgreSQL is the initial system of record. The entities below are conceptual groupings rather than finalized SQL tables; relationships, indexes, retention and migrations belong to detailed design.

Domain	Candidate records
Identity	users, sessions, instagram_connections, consent_choices
Analytics	instagram_posts, metric_observations, refresh_jobs, analysis_snapshots
AI	conversations, messages, approved_memories, model_calls
Content	media_assets, drafts, draft_versions, prohibited_phrases
Planning	planner_entries, reminders, publish_intents, publish_attempts
Competitor	competitor_accounts, competitor_posts, competitor_reviews
Discovery / news	ideas, idea_actions, evidence_links, official_source_items
Operations	background_jobs, audit_events, integration_errors

5. Data, evidence and lifecycle rules

5.1 Ownership and lineage

• Private records are associated with a Trellis user and checked at every read and mutation boundary; identities are not hardcoded (AUTH-02/04, MAINT-01).

• Provider identifiers remain separate from Trellis internal identifiers and include connection/account scope.

• An observation stores the data source, original observation/post time where known, last checked time and last changed/updated time when meaningful (DATA-02).

• Unavailable metrics are recorded as unknown/not accessible, never silently converted to zero; cross-account comparisons require compatible definitions and periods (COMP-03).

• Older history is preserved where legitimately available, with context-dependent relevance weighting rather than arbitrary exclusion (DATA-03).

• Private media bytes belong in an access-controlled storage service or repository; PostgreSQL retains identifiers, ownership, metadata and storage references. Vendor selection is deferred.

5.2 Conceptual record relationships

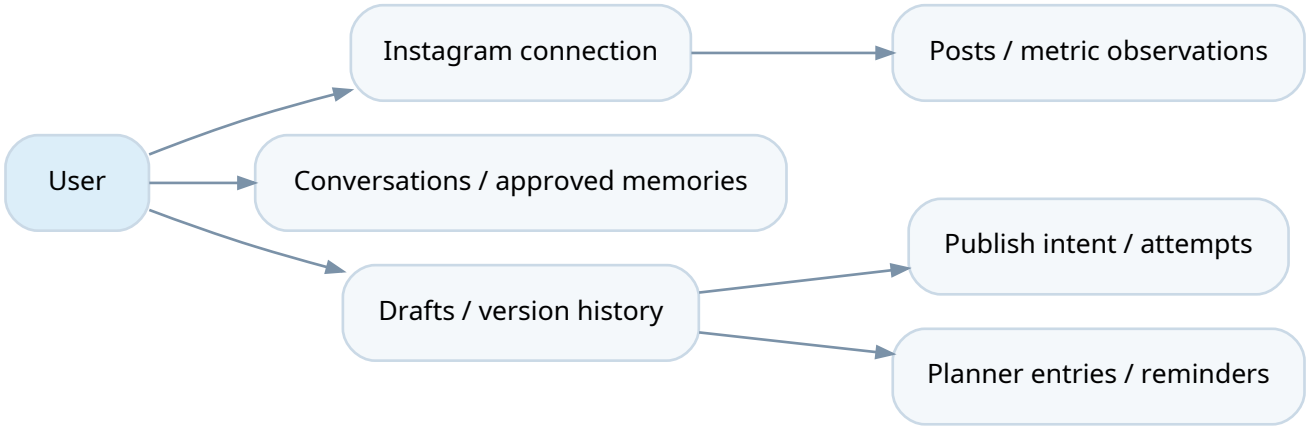


Figure 2. Simplified ownership and record lifecycle (not an ERD or final schema).

6. Representative request and processing flows

6.1 Dashboard opening and data refresh

Angular requests saved Dashboard information → Analytics returns available snapshots with freshness → server decides whether session-open analysis may reuse prior work or requires recomputation → authorized metric refresh may be queued according to preference and provider quota → new observations and timestamps are stored → UI receives accurate status. Opening the Dashboard must not block on unrelated remote calls (DATA-04/07, PERF-03).

6.2 Natural-language analytics question

Chat request → AI Orchestrator determines intent and relevant evidence → authorized analytics/query tool fetches matching records → Go performs deterministic ranking/comparison → AI composes a natural answer with scope, evidence and caveats → validator checks applicable numerical/source claims → reply returned. No unrestricted SQL or direct provider tokens are exposed to the model (AI-01/08).

6.3 Image-led draft creation and recovery

Image/text supplied → media validated and stored privately → selected caption context retrieved → AI creates hook, caption, CTA and hashtags → missing first-hand experience is an editable placeholder → prohibited-expression check → user edits → server persists version → browser displays Saved only after acknowledgment. Reload restores the latest confirmed version; failed writes remain visibly unsaved (CONT-01/08, PLAN-06, REL-01).

7. Manual publication lifecycle

A planned date is metadata. It creates reminder eligibility but never creates permission to publish. Each publication operates on the exact draft version approved by the currently authenticated user.

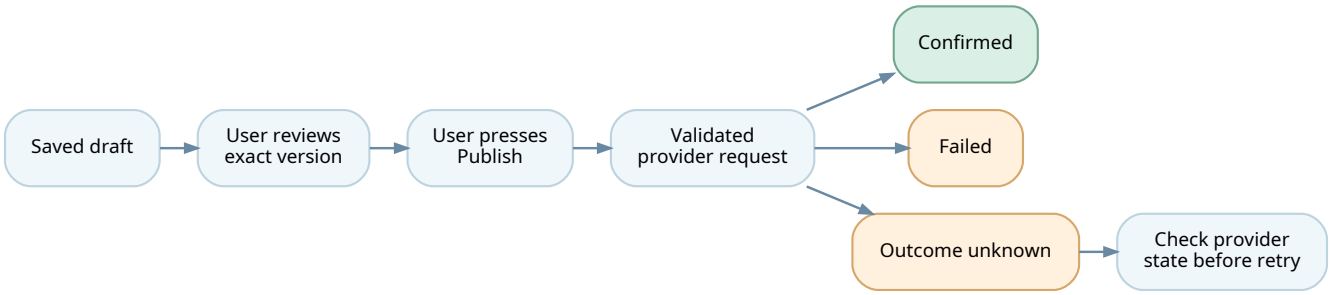


Figure 3. User-initiated publication states. No unattended repeat attempt or automatic rescheduling.

7.1 Safety invariants

• Enforce session identity, Instagram connection ownership, media and content validations, and an explicit action for the current draft version (PLAN-04, AUTH-08).

• Persist a publication intent and attempt record before/around the outbound request so ambiguous results can be investigated (REL-02).

• Deduplicate double-clicks and client retransmission. A provider timeout does not prove a failed publication.

• Do not silently retry an uncertain publish. Where available, reconcile provider state and require a fresh user-directed attempt (PLAN-03/05).

• Due-post reminders appear inside the app; dismissal of a toast does not clear pending tasks or unresolved errors (NOTIF-01/03).

8. AI boundaries, consistency and fallback

8.1 Tool and model boundaries

Layer	Design rule
Tool authorization	Allowlist read-only tools for analytics/evidence retrieval and approved draft-generation calls; every tool request has user scope.
Grounding	Separate observations from inference and unknowns; compare like-for-like metrics with deterministic functions.
Creator voice	Use selected relevant past captions for style while avoiding competitor voice imitation; unknown opinions become placeholders.
Guardrails	Check prohibited expressions and applicable calculation, attribution and image/product consistency conditions.
Free providers	Use compatible free-tier models only; preserve retrieval context for bounded fallback; no silent paid usage.
Memory	Use relevant recent turns and selective history; persist cross-chat memories only by explicit user instruction.

9. Security and privacy architecture

9.1 Trust boundaries

• Browser to API: authenticated HTTPS, request validation, anti-CSRF/session protections as appropriate, conservative upload constraints and rate limiting.

• User to user: server-enforced tenant ownership for records, chats, media, jobs and publication requests; direct object references do not grant access.

• App to Instagram: supported official authorization, least-privilege permissions, explicit account identity confirmation, encrypted at-rest tokens and restricted secrets access.

• App to AI: only the minimum necessary evidence is sent; do not expose authorization tokens, unrestricted queries or direct publishing operations.

• Disconnect: stop new collection immediately, invalidate/revoke access where supported, and present explicit, lawful retain/delete choices for permissible Instagram-related data (AUTH-05/07).

• Auditability: record important account-connection, consent, restriction and publishing actions without logging sensitive secrets.

> POLICY TO FINALIZE
> Trellis account deletion is distinct from disconnecting Instagram. Exact retention categories, deletion schedules, provider obligations and authentication implementation must be specified before release.

10. Reliability, jobs and operating design

Concern	Proposed mechanism
Work isolation	Make remote provider retrieval, eligible metric refresh and expensive analysis nonblocking; serve saved snapshots first.
Background processor	Initial worker processes persisted PostgreSQL jobs with ownership, status, attempts and last-error metadata; avoid a dedicated queue platform initially.
Retry policy	Bounded retries only for safe/idempotent operations; publishing ambiguity requires special reconciliation and user control.
Freshness & cost	Support user-selected refresh modes within quotas; deduplicate refreshes, reuse valid analysis and avoid unnecessary AI calls.
Observability	Structured logs, provider outcome/error classification, job progress, model fallback events, data freshness and publication history.
Degraded operation	Show saved data and explicit stale/error status if providers fail; do not fabricate metrics, news or AI completion.
Responsiveness	Do not block normal navigation, editing or swipes on AI calls; communicate progress for longer AI operations.

10.1 Deployment boundaries

Deployment environment, storage vendor, hosted authentication and service-level targets are not yet chosen. For the initial build, the modular backend can remain one deployable unit; operational job processing may run as a controlled process in that unit or as a separately started worker using the same codebase. This is a deployment choice, not a microservices commitment.

11. Architecture decision register

Decision	Proposed choice	State
ADR-001	Angular UI + modular Go monolith	Proposed
ADR-002	PostgreSQL as primary transactional and historical store	Proposed
ADR-003	Persisted PostgreSQL-backed jobs for safe asynchronous work	Proposed
ADR-004	Provider interfaces for Instagram, free AI, news and discovery	Proposed
ADR-005	Deterministic numerical analytics with source/timestamp lineage	Proposed
ADR-006	Immutable user-approved publishing versions; no unattended publishing	Proposed
ADR-007	Conversation context with explicit cross-chat memory permission	Proposed
ADR-008	Private media repository abstraction; vendor deferred	Proposed

12. Architectural risks and verification spikes

Priority	Risk / unknown	Required check
High	Instagram API eligibility, insights, publishing constraints	Verify supported account types, permissions, media formats, quotas and live responses.
High	Competitor data access and five-post rule	Validate permitted discovery, latest-five retrieval, skincare classification evidence and Growy-referenced coverage.
High	Free-tier AI models and fallback	Exercise tool use, multi-turn context, media understanding, failure recovery and hard no-paid constraint.
High	Privacy and retention	Define consent choices, credential revocation, deletion, categories and compliant evidence storage.
Medium	News/topics provenance	Choose compliant official/topic sources; verify dates, citations and change tracking.
Medium	iPad UX, autosave and performance	Measure saving, reload/recovery, touch interaction, load freshness and error states on iPad.

13. Requirements traceability

These mappings are examples of the principal architectural controls. They are not a replacement for a complete SRS-to-test traceability matrix.

Requirements	Architectural response
BR-01/02, DATA-01/08	Analytics module, evidence observations, snapshot refresh and deterministic comparisons.
BR-03, AI-01/08	Authorized AI tool orchestration, grounded context, output checks and bounded compatible fallback.
BR-04/06, CONT-01/09	Private media handling, voice context, structured draft package and prohibited-expression enforcement.
BR-07/08, PLAN-01/06, NOTIF-01/03	Dashboard planner, autosave versions, session reminders and explicit publication states.
BR-09/11, COMP/DISC/NEWS	Qualified competitor evidence, combined discovery/swipes and attributed official updates.
BR-12/14, DATA, PERF, UX	Quota-sensitive metric refresh, saved snapshots and responsive iPad-first browser workflows.
BR-13, AUTH-01/08, SEC-01	Trellis identity, Instagram account confirmation, tenant isolation and retention/disconnect flow.

14. Open decisions and approval gates

This proposal should not be treated as a verified integration design or approved final architecture until the following gates are resolved.

Gate	Owner / outcome needed
Product scope approval	Product Owner acknowledges manual publishing replacing unattended scheduling and agrees on session-based analysis behavior.
Instagram & competitor feasibility	Technical lead records tested permission/access matrix, provider quotas, missing-data behavior and compliant sources.
Authentication & storage	Choose session/auth implementation, credential management, private asset storage and account-disconnection workflow.
AI feasibility	Identify tested free model/provider(s), fallback eligibility, prohibited-content checks and memory retention behavior.
Quantitative NFRs	Set measurable targets for iPad responsiveness, autosave, freshness, quotas and availability.
Detailed design	Produce API contracts, finalized ERD, state-machine transitions, job semantics, component/module interfaces and threat model.

Gate	Owner / outcome needed
Test planning	Trace SRS acceptance scenarios T-01 to T-10 to automated/manual verification and review evidence.

14.1 Recommended immediate technical investigations

• Spike A: Connect a test Instagram account through a supported flow and enumerate permitted metrics, history and publishing capabilities.

• Spike B: Check whether a compliant source provides the competitor latest-five-post evidence and comparable measures required by COMP-01/04.

• Spike C: Prototype one grounded AI analytics query and one image-led draft using candidate free models and bounded failure handling.

• Spike D: Validate one draft autosave/reload cycle and reminder-to-manual-publish sequence on an iPad browser.

STATUS: PROPOSED FOR REVIEW
The architecture is internally consistent with the BRD/SRS direction but deliberately leaves deployment, vendor choice, access feasibility, precise quantitative targets and legal retention controls unresolved. Record approvals and changes in architecture decision records before implementation.

References and authority

R1. Trellis Business Requirements Document, v1.0, 22 October 2026, consolidated working baseline for review.

R2. Trellis Software Requirements Specification, v1.0, 22 October 2026, consolidated working baseline for review.

Authority note. Requirements identifiers refer to R1/R2. Suggested components, candidate entities, jobs and ADRs are architecture proposals, not approved requirements or verified external capabilities.